K-Nearest Neighbour (KNN)

A Beginner's Introduction to Distance-Based Learning

Machine Learning

What is K-Nearest Neighbour?

Definition

K-Nearest Neighbour (KNN) is a **supervised machine learning algorithm** that makes predictions based on the observations that are closest to a new observation.

- It can be used for **classification**.
- It can be used for **regression**.
- KNN is called a **distance-based** method.
- KNN is often described as a **lazy learning** or **instance-based** algorithm.

Basic Idea

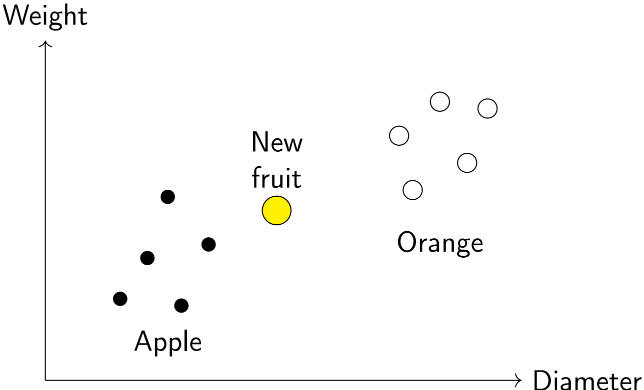
Tell me who your nearest neighbours are, and I will predict who you are.

A Simple Real-Life Analogy

Suppose we want to identify whether a fruit is an **Apple** or an **Orange**.
We measure:

- Weight
- Diameter

If a new fruit is surrounded mostly by apples, we classify it as an apple.



Why Do We Need KNN?

Consider a dataset containing observations for which the class is already known.

Problem

A new observation arrives, but its class is unknown.

KNN asks:

1. Which training observations are closest to the new observation?
2. What are their classes?
3. Which class occurs most frequently among the nearest neighbours?

Key Principle

Nearby observations in the feature space are assumed to have similar outcomes.

Supervised Learning Perspective

Training Data → Calculate Distances → Find K Nearest → Make Prediction

Training Data → Distance → Neighbours → Prediction

What is a Feature?

A **feature** is a measurable characteristic used to describe an observation.

Example: Student Dataset

Student	Study Hours	Attendance
A	2	60
B	5	80
C	7	90
D	3	70

Here:

$$X_1 = \text{Study Hours}$$

$$X_2 = \text{Attendance}$$

Each student is represented by a point:

Feature Space

A **feature space** is the mathematical space formed by the features used to represent observations.
For two features:

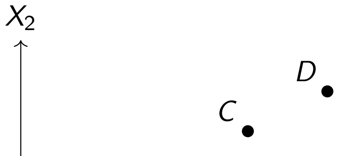
$$\mathrm{x} = (x_1, x_2)$$

For p features:

$$\mathrm{x} = (x_1, x_2, \ldots, x_p)$$

Important Idea

Each observation corresponds to a point in the feature space.



Distance-Based Learning

KNN relies on the concept of **distance**.

Central Question

How close is a new observation to each observation in the training dataset?

If two observations have a small distance:

$$d(\mathrm{x}_i, \mathrm{x}_j) \approx 0$$

they are considered similar according to the selected distance measure.

KNN Principle

Smaller distance $\Rightarrow$ greater similarity.

Euclidean Distance

For two observations:

$$\mathbf{x} = (x_1, x_2)$$

$$\mathbf{y} = (y_1, y_2)$$

the Euclidean distance is:

$$\boxed{d(\mathbf{x}, \mathbf{y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}}$$

For p features:

$$\boxed{d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{j=1}^{p} (x_j - y_j)^2}}$$

where:

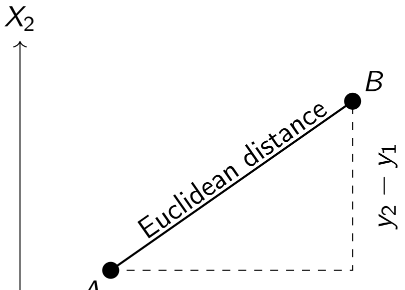
Euclidean Distance: Geometric Interpretation

For two points:

$$A = (x_1, y_1)$$

$$B = (x_2, y_2)$$

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$



Euclidean Distance: Numerical Example

Let:

$$A = (2, 3)$$

$$B = (5, 7)$$

Then:

$$d(A, B) = \sqrt{(5-2)^2 + (7-3)^2}$$

$$= \sqrt{3^2 + 4^2}$$

$$= \sqrt{9 + 16}$$

$$= \sqrt{25}$$

Manhattan Distance

Manhattan distance measures distance by adding the absolute differences between coordinates. For p features:

$$\boxed{d(\mathrm{x}, \mathrm{y}) = \sum_{j=1}^{p} |x_j - y_j|}$$

For two features:

$$d(\mathrm{x}, \mathrm{y}) = |x_1 - y_1| + |x_2 - y_2|$$

Example

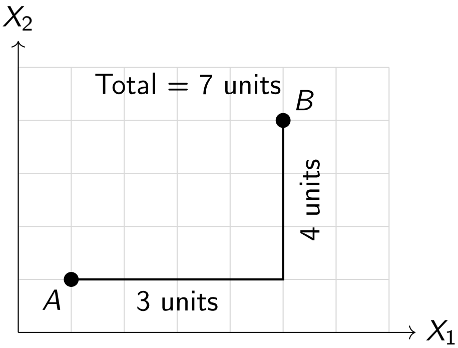
For:

$$A = (2, 3), \qquad B = (5, 7)$$

$$d(A, B) = |5 - 2| + |7 - 3|$$

Manhattan Distance: Intuition

Manhattan distance can be visualized as movement along horizontal and vertical paths.



Minkowski Distance

Minkowski distance provides a general family of distance measures.

$$\boxed{d(\mathrm{x}, \mathrm{y}) = \left(\sum_{j=1}^{p} |x_j - y_j|^q \right)^{1/q}}$$

where q determines the type of distance.

q	Distance
1	Manhattan distance
2	Euclidean distance
$q > 0$	General Minkowski distance

$$q = 1 \Rightarrow L_1 \qquad q = 2 \Rightarrow L_2$$

Comparison of Common Distance Measures

Distance	Formula	Common Name
L_1	$\sum \lvert x_j - y_j \rvert$	Manhattan
L_2	$\sqrt{\sum (x_j - y_j)^2}$	Euclidean
L_q	$(\sum \lvert x_j - y_j \rvert^q)^{1/q}$	Minkowski

Practical Point

The choice of distance measure can influence which observations become the nearest neighbours.

KNN Classification

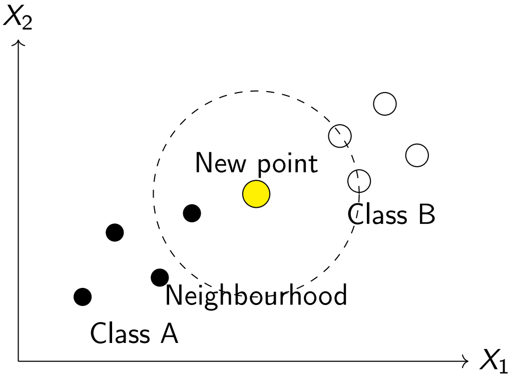
In classification, KNN predicts a **categorical class**.
Suppose the possible classes are:

$$\mathcal{C} = \{\mathsf{A}, \mathsf{B}\}$$

For a new observation x_0:

1. Calculate its distance from every training observation.
2. Sort the observations by distance.
3. Select the K closest observations.
4. Count their class labels.
5. Assign the majority class to x_0.

KNN Classification: Visual Idea



For an appropriate K, the majority of nearby observations determines the predicted class.

What Does K Mean?

K is the number of nearest training observations used to make the prediction.

Examples

- $K = 1$: use the nearest observation.
- $K = 3$: use the three nearest observations.
- $K = 5$: use the five nearest observations.
- $K = 10$: use the ten nearest observations.

$$\boxed{K = \text{Number of neighbours considered}}$$

The value of K is an important **hyperparameter**.

Numerical Example: Problem Setup

Consider the following training data.

Observation	X_1	X_2	Class
A	1	1	Red
B	2	2	Red
C	3	1	Red
D	6	5	Blue
E	7	6	Blue
F	8	5	Blue

New observation:

$$\boxed{X = (4, 3)}$$

Let:

$$\boxed{K = 3}$$

Step 1: Calculate Distances

Use Euclidean distance.

$$d(X, A) = \sqrt{(4-1)^2 + (3-1)^2} = \sqrt{13} \approx 3.606$$

$$d(X, B) = \sqrt{(4-2)^2 + (3-2)^2} = \sqrt{5} \approx 2.236$$

$$d(X, C) = \sqrt{(4-3)^2 + (3-1)^2} = \sqrt{5} \approx 2.236$$

$$d(X, D) = \sqrt{(4-6)^2 + (3-5)^2} = \sqrt{8} \approx 2.828$$

Step 1: Calculate Remaining Distances

$$d(X, E) = \sqrt{(4-7)^2 + (3-6)^2} = \sqrt{18} \approx 4.243$$

$$d(X, F) = \sqrt{(4-8)^2 + (3-5)^2} = \sqrt{20} \approx 4.472$$

Therefore:

Observation	Distance
A	3.606
B	2.236
C	2.236
D	2.828
E	4.243
F	4.472

Step 2: Sort the Distances

Rank	Observation	Distance
1	B	2.236
2	C	2.236
3	D	2.828
4	A	3.606
5	E	4.243
6	F	4.472

Since:

$$K = 3$$

we select:

$$\boxed{B, \ C, \ D}$$

Their classes are:

Step 3: Majority Voting

The three nearest neighbours are:

Neighbour	Class
B	Red
C	Red
D	Blue

Class counts:

$$N_{\text{Red}} = 2$$

$$N_{\text{Blue}} = 1$$

Therefore:

$$\boxed{\hat{y} = \text{Red}}$$

Final Prediction

Complete KNN Classification Procedure

1. Choose K
2. Calculate distances
3. Sort distances
4. Select K nearest
5. Count class labels
6. Majority vote

Choosing the Value of K

The value of K strongly affects KNN.

Small K

The model focuses strongly on local observations.

Large K

The model considers a broader neighbourhood.

A common approach is to select K using validation or cross-validation.

$$\boxed{K^* = \arg\min_K \text{Validation Error}(K)}$$

The best K depends on the dataset.

Effect of Small K

For example:

$$K = 1$$

Only one neighbour determines the prediction.

- Very sensitive to individual observations.
- Sensitive to noise.
- Can produce a highly irregular decision boundary.
- Low bias but potentially high variance.

Risk

A noisy training observation can strongly influence the prediction.

Effect of Large K

When K becomes very large:

- More observations influence each prediction.
- Local patterns become less important.
- The model becomes smoother.
- Different classes may be mixed together.
- The model can become influenced by the overall class distribution.

Risk

An excessively large K may cause underfitting.

Small K versus Large K

	Small K	Large K
Neighbourhood	Local	Broad
Sensitivity to noise	High	Lower
Variance	Higher	Lower
Bias	Lower	Higher
Decision boundary	Irregular	Smoother
Risk	Overfitting	Underfitting

$\boxed{\text{Choose } K \text{ to balance bias and variance}}$

Why Odd Values of K Are Often Used

Consider binary classification:

$$\{\text{Class A}, \text{Class B}\}$$

If:

$$K = 4$$

a vote can result in:

$$2 \text{ votes for A}, \qquad 2 \text{ votes for B}$$

This produces a tie.
For:

$$K = 5$$

a possible majority is:

Why Feature Scaling Matters

Suppose we have two features:

$$X_1 = \text{Age}$$

$$X_2 = \text{Annual Income}$$

Age might range from:

$$18 \text{ to } 80$$

while income might range from:

$$20,000 \text{ to } 2,000,000$$

The larger numerical scale can dominate the distance calculation.

Key Point

KNN is sensitive to the scale of features because it directly uses distances.

Example of the Scaling Problem

Suppose:

$$A = (20, 50000)$$

$$B = (25, 51000)$$

Euclidean distance:

$$d(A, B) = \sqrt{(25 - 20)^2 + (51000 - 50000)^2}$$

$$= \sqrt{25 + 1,000,000}$$

The second feature dominates the distance.

Solution

Scale the features before applying KNN.

Standardization

One common scaling method is standardization.

$$\boxed{z = \frac{x - \mu}{\sigma}}$$

where:

- x = original value
- μ = mean of the feature
- σ = standard deviation
- z = standardized value

After standardization, features are placed on a comparable scale.

Practical Workflow

Training Data $\rightarrow$ Fit Scaler $\rightarrow$ Transform Training and Test Data $\rightarrow$ KNN

Min-Max Scaling

Another common method is Min-Max scaling.

$$\boxed{x' = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}}$$

Usually:

$$0 \leq x' \leq 1$$

Example

If:

$$x = 60, \quad x_{\text{min}} = 20, \quad x_{\text{max}} = 100$$

then:

$$x' = \frac{60 - 20}{100 - 20}$$

Why Weighted KNN?

Standard KNN gives every selected neighbour equal voting importance.
But consider:

$$d_1 = 0.5, \qquad d_2 = 2.0$$

The first neighbour is much closer.
Weighted KNN gives greater importance to closer observations.

Basic Idea

Closer neighbour $\Rightarrow$ larger weight.

One common weighting scheme is:

$$\boxed{w_i = \frac{1}{d_i}}$$

where d_i is the distance from the new observation to neighbour i.

Weighted KNN Classification

For class c, a weighted vote can be written as:

$$\boxed{V(c) = \sum_{i \in N_K} w_i I(y_i = c)}$$

where:

- N_K = set of K nearest neighbours
- w_i = weight of neighbour i
- y_i = class of neighbour i
- $I(y_i = c) = 1$ if $y_i = c$, otherwise 0

The predicted class is:

$$\boxed{\hat{y} = \arg\max_c V(c)}$$

Weighted KNN: Numerical Example

Suppose three neighbours have:

Neighbour	Distance	Class
A	1	Red
B	2	Blue
C	4	Blue

Using:

$$w_i = \frac{1}{d_i}$$

we obtain:

$$w_A = 1, \qquad w_B = 0.5, \qquad w_C = 0.25$$

Weighted votes:

$$V(\text{Red}) = 1$$

KNN for Regression

KNN can also predict a **continuous numerical value**.
Instead of majority voting, we usually calculate the average target value of the K nearest neighbours.

$$\boxed{\hat{y} = \frac{1}{K} \sum_{i \in N_K} y_i}$$

where:

- N_K = set of K nearest neighbours
- y_i = target value of neighbour i
- $\hat{y}$ = predicted value

KNN Regression: Numerical Example

Suppose the three nearest neighbours have house prices:

$$40 \text{ lakh}, \quad 45 \text{ lakh}, \quad 50 \text{ lakh}$$

For:

$$K = 3$$

the prediction is:

$$\hat{y} = \frac{40 + 45 + 50}{3}$$

$$= \frac{135}{3}$$

$$\boxed{\hat{y} = 45 \text{ lakh}}$$

Weighted KNN Regression

Weighted KNN can also be used for regression.

$$\boxed{\hat{y} = \frac{\sum_{i \in N_K} w_i y_i}{\sum_{i \in N_K} w_i}}$$

with:

$$w_i = \frac{1}{d_i}$$

or another decreasing function of distance.

Interpretation

Closer observations contribute more strongly to the prediction.

KNN Algorithm

Input

Training dataset, new observation x_0, value of K, and a distance measure.

1. Store the training observations and their labels.
2. Choose K.
3. Calculate the distance between x_0 and every training observation.
4. Sort the distances.
5. Select the K observations with the smallest distances.
6. For classification, perform majority voting.
7. For regression, calculate the average or weighted average.
8. Return the prediction.

KNN Algorithm: Mathematical Form

Let the training dataset be:

$$D = \{(\mathrm{x}_i, y_i)\}_{i=1}^{n}$$

For a new observation x_0, calculate:

$$d_i = d(\mathrm{x}_0, \mathrm{x}_i), \qquad i = 1, \ldots, n$$

Let:

$$N_K(\mathrm{x}_0)$$

denote the indices of the K smallest distances.

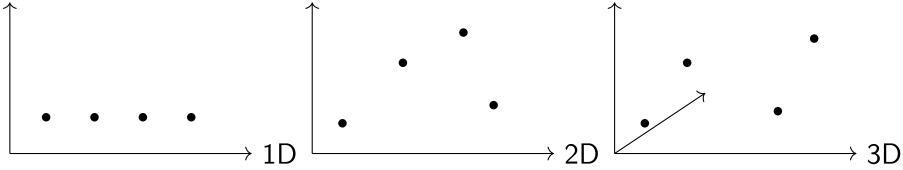
For classification:

$$\boxed{\hat{y} = \arg\max_{c} \sum_{i \in N_K(\mathrm{x}_0)} I(y_i = c)}$$

For regression:

What is the Curse of Dimensionality?

KNN depends on the concept of distance.
As the number of features increases, the feature space becomes increasingly large and sparse.



Problem

Distance-based similarity can become less informative in very high-dimensional spaces.

High Dimensional Data and KNN

Suppose a dataset contains:

$$p = 2$$

features.

It is possible to visualize the observations on a plane.

But if:

$$p = 100$$

we have a 100-dimensional feature space.

- Data become sparse.
- Distances can become less discriminative.
- More data may be required.
- Computation can increase.
- Irrelevant features can distort distances.

Possible Solutions

Is KNN Expensive?

KNN has a different computational characteristic from many models.

Training

Usually very little model fitting is required.

$$\text{Training} \approx \text{Store the training data}$$

Prediction

Distances must be calculated between the new observation and training observations.

$$\text{Prediction} \approx \text{Many distance calculations}$$

Key Idea

KNN is often computationally inexpensive during training but potentially expensive during prediction.

Important KNN Hyperparameters

Hyperparameter	Purpose
K	Number of neighbours
Distance metric	Defines similarity/distance
Weights	Equal or distance-based weighting
Feature scaling	Controls influence of feature scales
Algorithm	Strategy for neighbour search

Most Important

The choice of K, distance metric, and feature scaling can substantially affect KNN performance.

Advantages of KNN

- Simple and intuitive.
- Easy to understand.
- Can be used for classification and regression.
- No strong parametric distributional assumption is required.
- Naturally handles multiclass classification.
- Can model nonlinear decision boundaries.
- Useful as a baseline model.

Main Strength

KNN can capture local patterns without assuming a specific functional form.

Disadvantages of KNN

- Prediction can be computationally expensive for large datasets.
- Sensitive to the choice of K.
- Sensitive to feature scaling.
- Sensitive to irrelevant features.
- Sensitive to noisy observations.
- Performance can degrade in high dimensions.
- Requires storing the training dataset.

Important

Good preprocessing and a suitable choice of K are particularly important for KNN.

Important Assumptions and Considerations

KNN does not require assumptions such as normality in the same way as some classical statistical models.
However, it relies on important practical ideas:

- The distance measure should represent meaningful similarity.
- Nearby observations should tend to have similar outcomes.
- Features should be appropriately scaled when necessary.
- Irrelevant variables should be avoided or controlled.
- The choice of K should be appropriate.

Applications of KNN

KNN can be applied in many areas:

- Pattern recognition
- Image classification
- Recommendation systems
- Medical classification
- Customer segmentation
- Text classification
- Anomaly detection
- Handwritten digit recognition

General Principle

Whenever similarity between observations is meaningful, KNN can be considered as a possible method.

Where Does KNN Fit?

1. Prepare the dataset.
2. Split into training and testing data.
3. Scale features when appropriate.
4. Select K and distance metric.
5. Store the training observations.
6. Predict using nearest neighbours.
7. Evaluate performance.

KNN Classification vs KNN Regression

	Classification	Regression
Target	Categorical	Continuous
Prediction	Class label	Numerical value
Main operation	Majority vote	Mean/weighted mean
Example	Disease class	House price
Output	$\hat{y} \in \mathcal{C}$	$\hat{y} \in \mathbb{R}$

KNN Compared with Other Models

Property	KNN	Decision Tree	Linear Model
Learning style	Instance-based	Tree-based	Parametric
Distance required	Yes	No	No
Scaling important	Usually	Usually not	Often
Nonlinear patterns	Yes	Yes	Limited without transformation
Interpretability	Moderate	High	High
Prediction cost	Can be high	Usually lower	Usually low

Common Mistakes When Using KNN

1. Choosing K arbitrarily without validation.
2. Forgetting to scale numerical features.
3. Including irrelevant features.
4. Using an unsuitable distance measure.
5. Ignoring class imbalance.
6. Evaluating only on training data.
7. Applying preprocessing using information from the test set.
8. Using very high-dimensional raw data without considering dimensionality.

Good Practice

Preprocessing and model selection must be performed without leaking information from the test set.

Classroom Practice Problem 1

Consider:

Point	X_1	X_2	Class
A	1	2	Red
B	2	3	Red
C	3	3	Blue
D	6	5	Blue
E	7	6	Blue

New point:

$$X = (4, 4)$$

Use:

$$K = 3$$

and Euclidean distance.

1. Calculate all distances

Classroom Practice Problem 2

A KNN regression problem has the following nearest neighbours:

Distance	Target
1	20
2	30
3	40
4	50

Using:

$$K = 3$$

calculate:

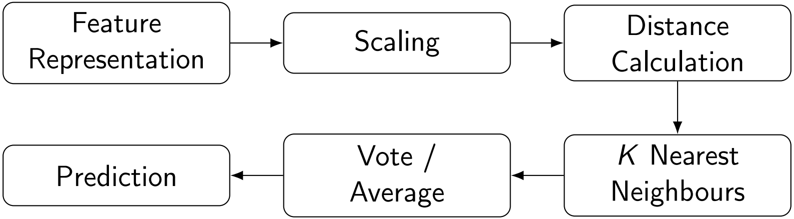
1. The ordinary KNN regression prediction.
2. The weighted prediction using:

$$w_i = \frac{1}{d_i}$$

KNN: Key Concepts

- KNN is a supervised, distance-based learning method.
- It can perform classification and regression.
- K determines how many neighbours are considered.
- Euclidean, Manhattan, and Minkowski distances are common choices.
- Classification commonly uses majority voting.
- Regression commonly uses averaging.
- Weighted KNN gives greater importance to closer observations.
- Feature scaling is important when features have different scales.
- Small K can lead to high variance.
- Large K can lead to high bias.
- High-dimensional data can create the curse of dimensionality.

KNN: The Complete Picture



$\boxed{\text{Represent} \rightarrow \text{Measure Distance} \rightarrow \text{Find Neighbours} \rightarrow \text{Aggregate} \rightarrow \text{Predict}}$

K-Nearest Neighbour

$\boxed{\text{Similarity through Distance}}$

Classification $\rightarrow$ Majority Vote

Regression $\rightarrow$ Average / Weighted Average